GVP MONITORING USING IMAGE SEGMENTATION

Team Phoenix:

Krishna Kaundinya Peddibhotla

Nikhith Reddy K

Phanindra Gollapalli

Krithik Khambhampatti

Advisor: Dr. Sachin Chaudari

Associate Professor

IIIT-HYD

Date: 17.11.2025

Project Statement

- A QIDK-based GVP monitoring solution is to be developed to determine parameters such as the area and amount of garbage, dumping patterns, etc.
- The camera module also is to be integrated with the QIDK Kit to enable real-time image capture and analysis with reduced latency

MOTIVATION

- Areas prone to garbage dumping are not only harmful for the environment but also for the people living in the vicinity.
- Bad odor, breeding spots for insects, etc. are all serious issues that need to be addressed immediately
- If we plan to develop our city into a smart city, we need to address the issue of garbage dumping
- Dumping patterns, collection schedules, etc need to be studied to solve the problem of garbage

Methodology

Lab 1: Learnt about the project and label-studio

Lab 2-3: Annotated images

Lab 4: Trained the models in Qualcomm AI Hub

Lab 5: Made a GUI to show the model’s working

Lab 6: Uploaded best model(YOLOv11) in the QIDK Kit

Lab 6: Retrained model using our own custom dataset

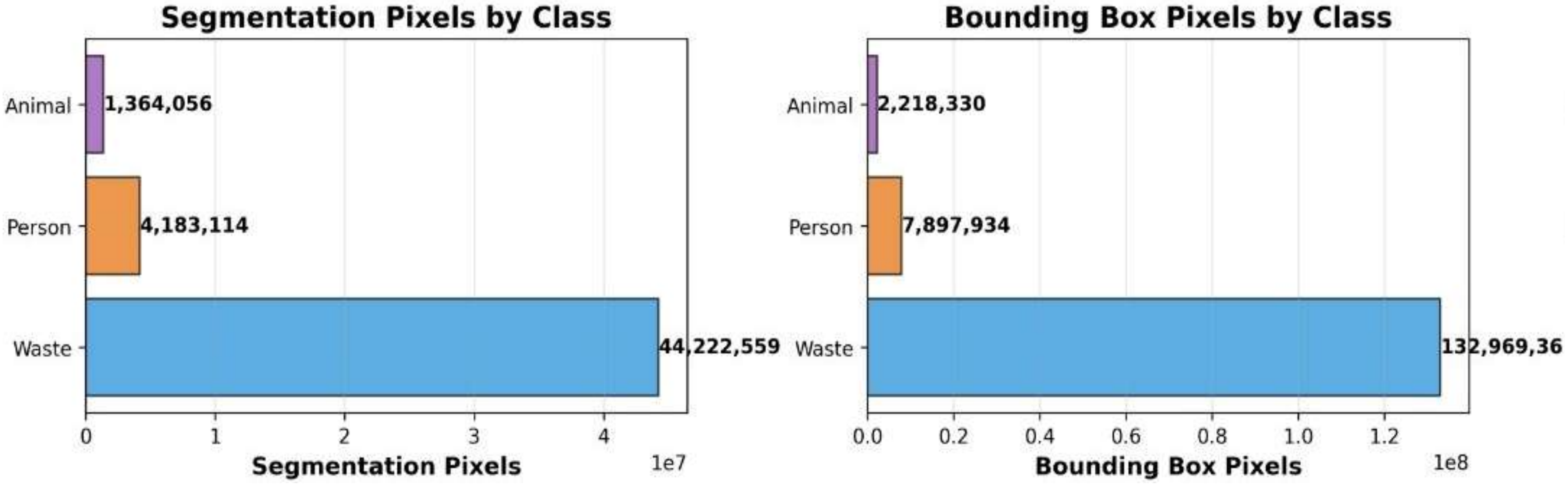
Lab 7: Added coverage area, pixel count and processing time pictures

Lab 8: Trained and compared Unet model

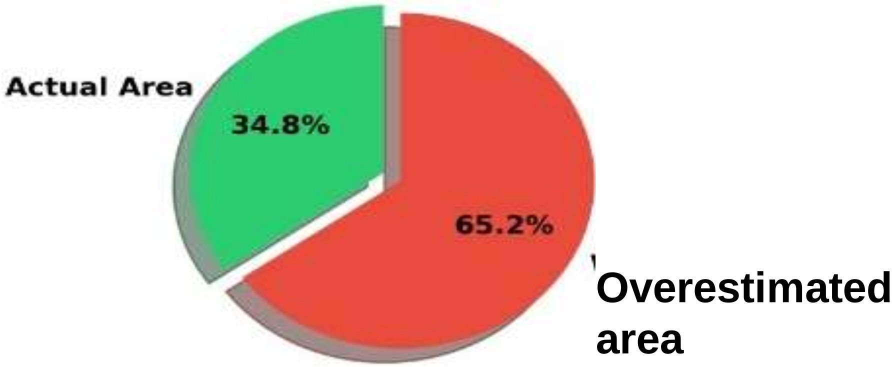
Lab 8: Compared metrics of all models

Lab 9: Uploaded model on Raspberry Pi 4 and compared with the QIDK Kit. Also, we have deployed the kit in the field.

Image Segmentation over Object Detection



For bounding box method:



This analysis proves that image segmentation method is better than bounding boxes method.

The bounding boxes method overestimates the area by 65.2%

Dataset

Our dataset contains annotated images from 3 different sources:

- **The dataset provided by the TA (1500+):** These images focussed on two garbage dumping spots in the city captured using CCTV footage
- **The dataset collected by us from inside and outside the campus(100+):** To avoid overfitting due to only two locations in the above dataset, we built our own dataset.
- **The dataset made up of images from Google (100+):** Since the above dataset is focused only on our campus, we used Google to obtain garbage dumping spots from all over the country to better train our model.

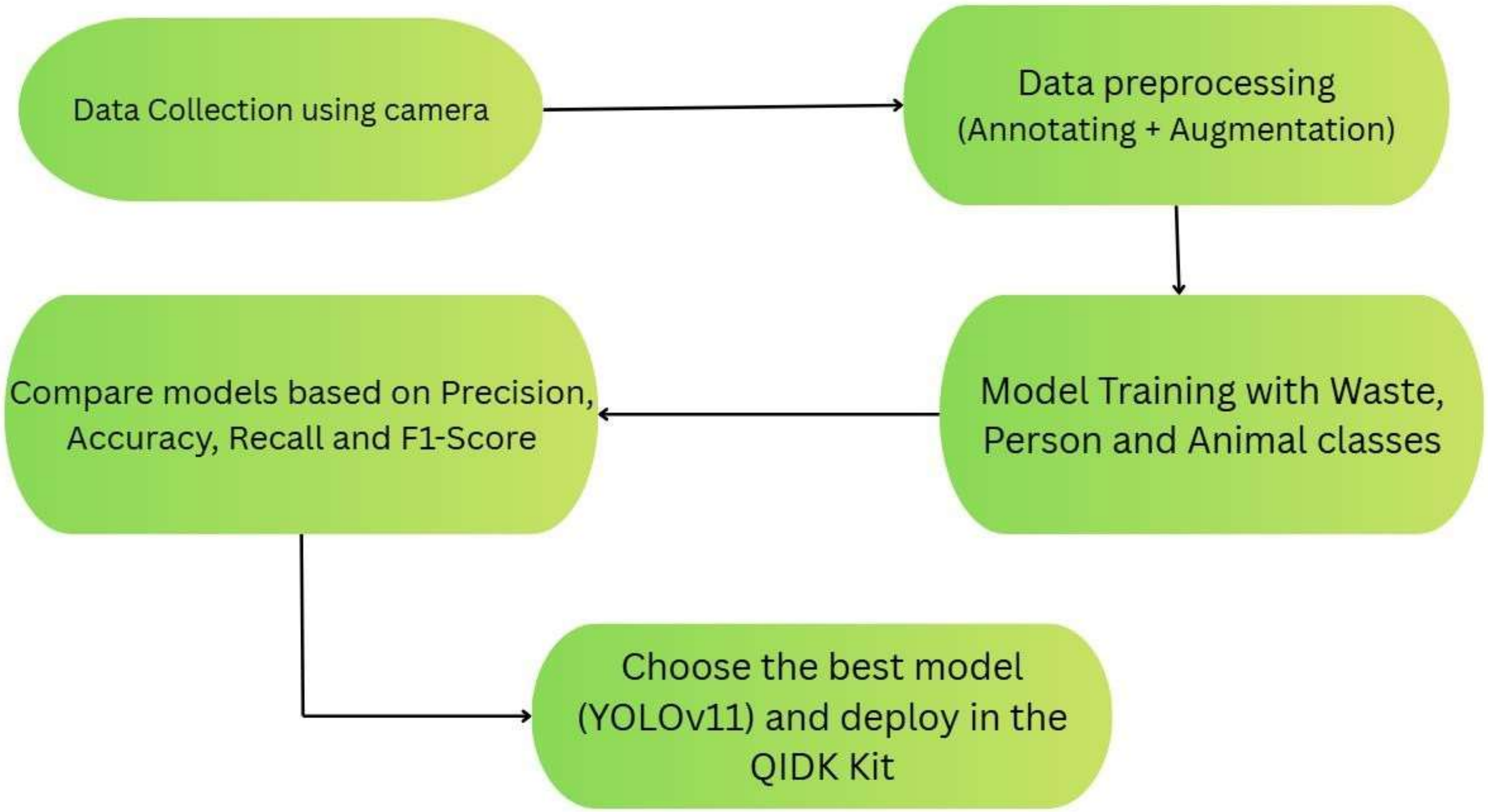
Annotation into 3 classes (Waste, Person and Animal)

Models and Framework

We used the **Qualcomm AI Hub** platform to get to know all the below models. The models trained were:

- **YOLOv8 Segmentation model** - Combines object detection and segmentation, extremely fast and optimized for real-time inference on edge devices
- **FCN ResNet50 model -** One of the most stable and classical semantic segmentation architectures, includes deep feature extraction
- **DDRNet23 - Slim model -** Designed for real-time, lightweight segmentation and suitable for edge devices with low compute and less power usage
- **YOLOv11 Segmentation model -** High-accuracy segmentation with modern optimizations suited to Qualcomm NPUs.
- **Unet model -** Accurate mask generation in scenarios with irregular shapes(like garbage spread) and limited labeled samples.

Block Diagram for models



Parameters Considered for Model Comparison

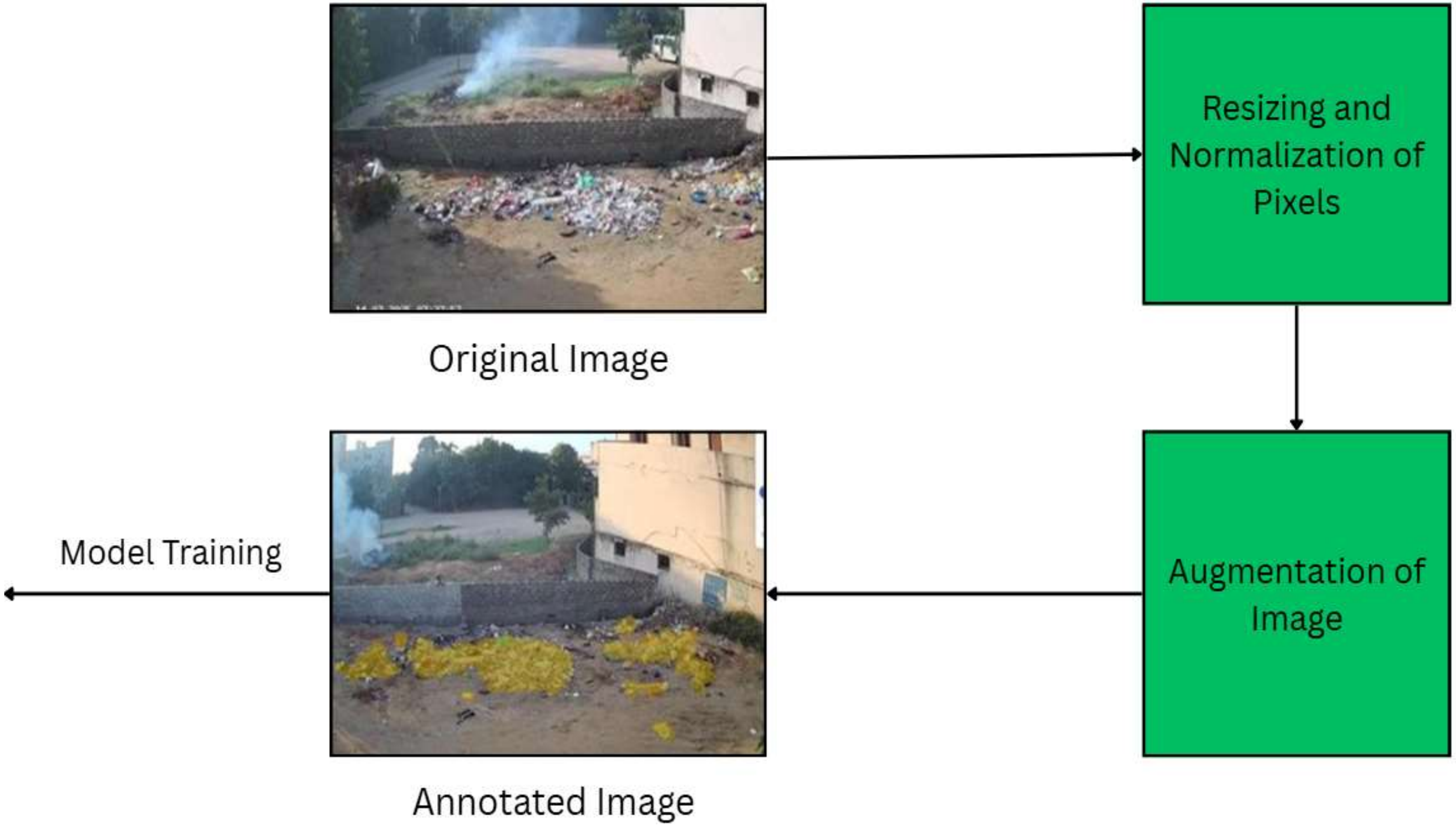
- We considered the following parameters for choosing the best performing model:
  - F1-Score (2* Precision* Recall / (Precision + Recall))
  - Mean IoU

Analysis of Parameters (Mean IoU)

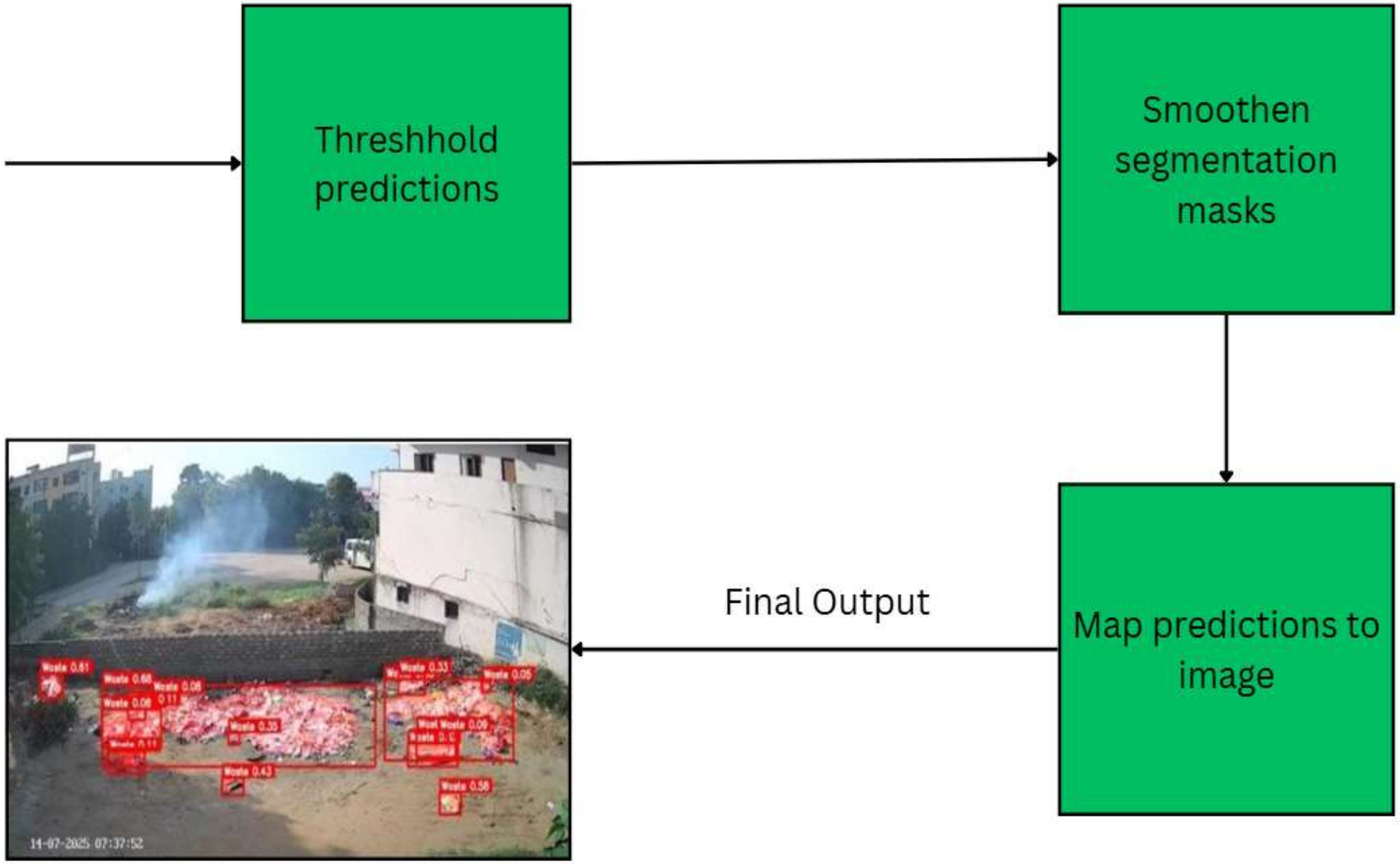
Model	Mean IoU	F1-score
YOLOv11	**0.75**	**0.69**
YOLOv8	0.72	0.69
DDRNet23 – Slim	0.50	0.66
FCN ResNet50	0.61	0.66
Unet	0.49	0.60

Because Yolo v11 had better results compared to other models, we decided to upload it in the QIDK Kit

Pre-processing of Data

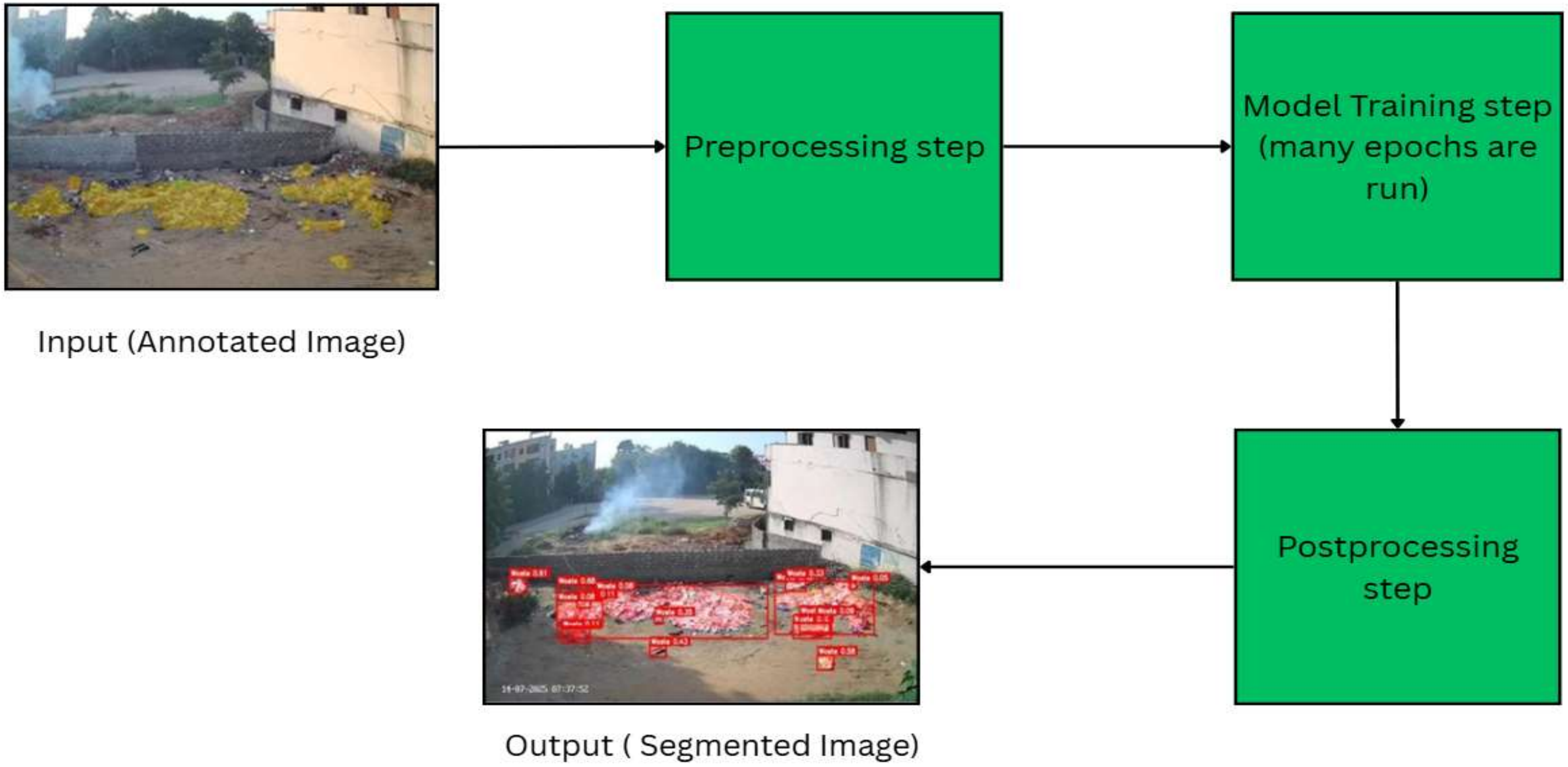


Post-Processing of Data



Demonstration of GUI

Workflow of the model



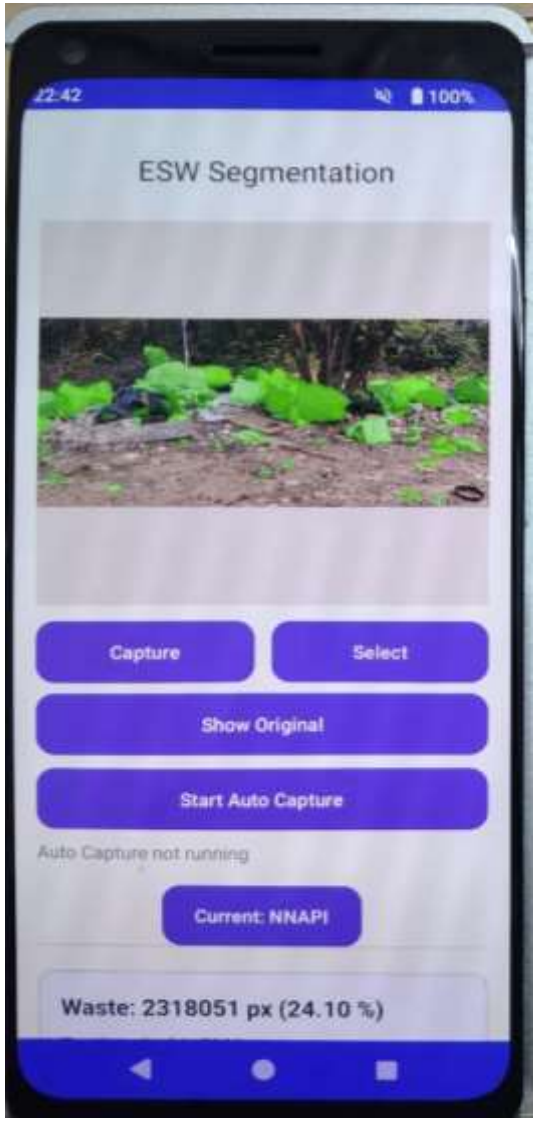
Functionality of the Android App

The YOLOv11 model was uploaded into the app

In the app, you can take a photo or use one from the gallery

You can view the segmented image, coverage and pixel count

The app shows the processing time



NPU runtime of the Android App

The android app has the feature of showing the runtime of each image.

This runtime refers to the time taken by the model in the lab to process the image

Neural processing unit (NPU) is a specialized AI accelerator in embedded devices like the QIDK Kit to quicken the processing

It acts like the CPU and processes the images

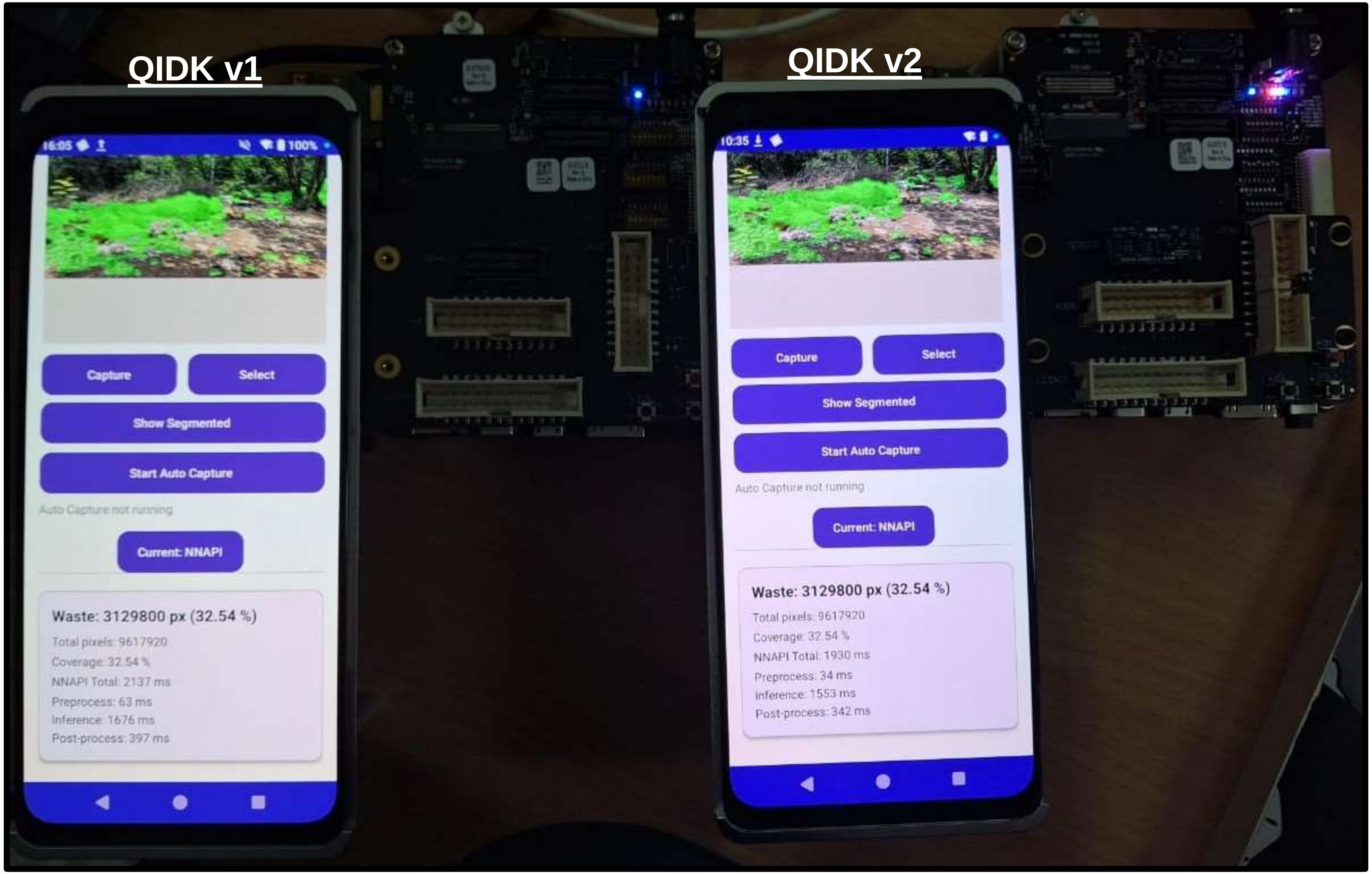
Comparison of processing times

We observed that the NPU achieves approximately **100 ms** faster processing time compared to the CPU. Below is an example where the same image was provided as input to both CPU and NPU for comparison. In the Raspberrypi 4, the inference CPU time is high (around **18 s**) because multiple threads run at once. The wall time shows the actual processing time. (around **13s**)

	QIDK (CPU)	QIDK (NPU)	Raspi 4
Processing time	**2206 ms**	**2076 ms**	**17.054 s**

The time of the Raspi 4 is significantly higher as the device runs on comparatively older CPU models, and does not have optimised ML kernels like the QIDK

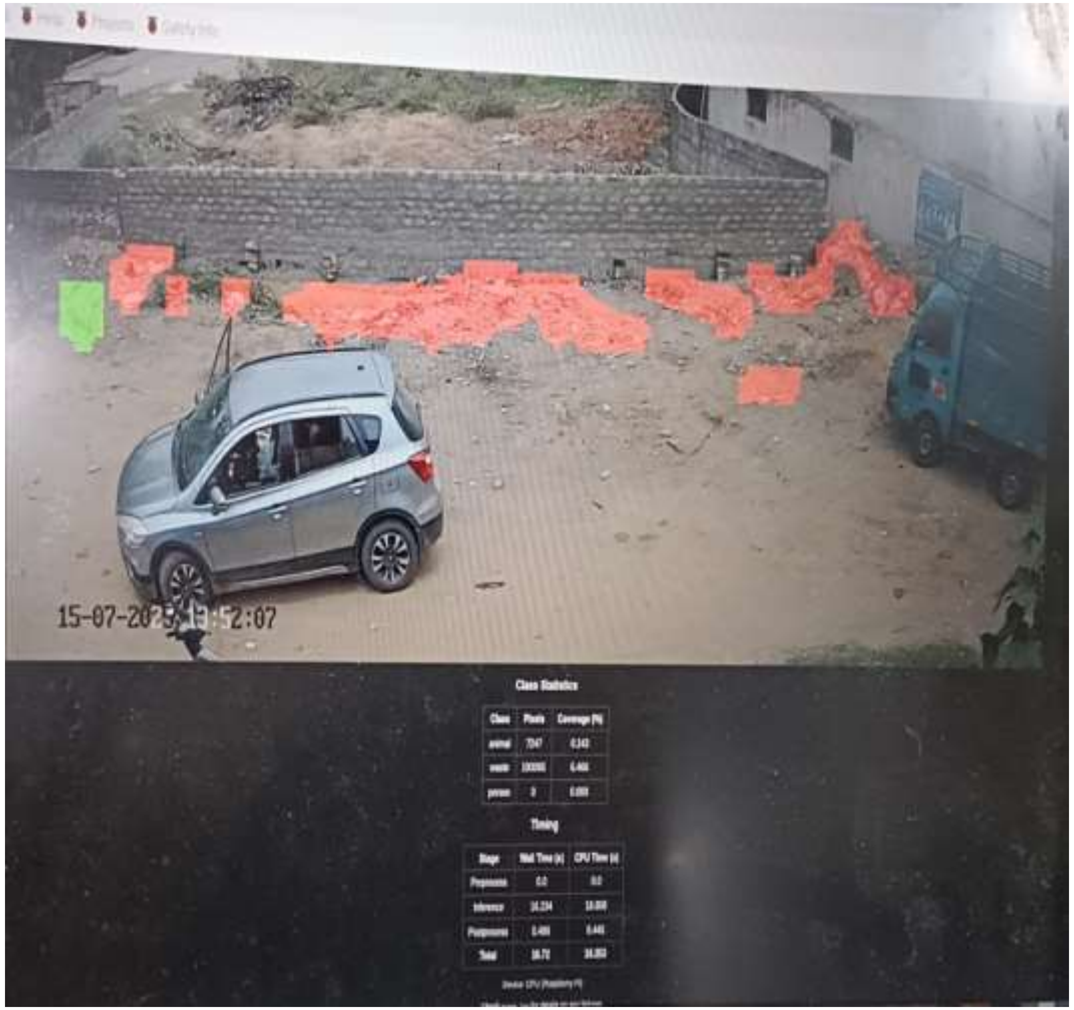
QIDK v1 VS QIDK v2



On average, the QIDK v2 Kit's NPU is able to segment images **~200 ms** faster when compared to the QIDK v1 Kit's NPU

Functionality of the RaspberryPi 4

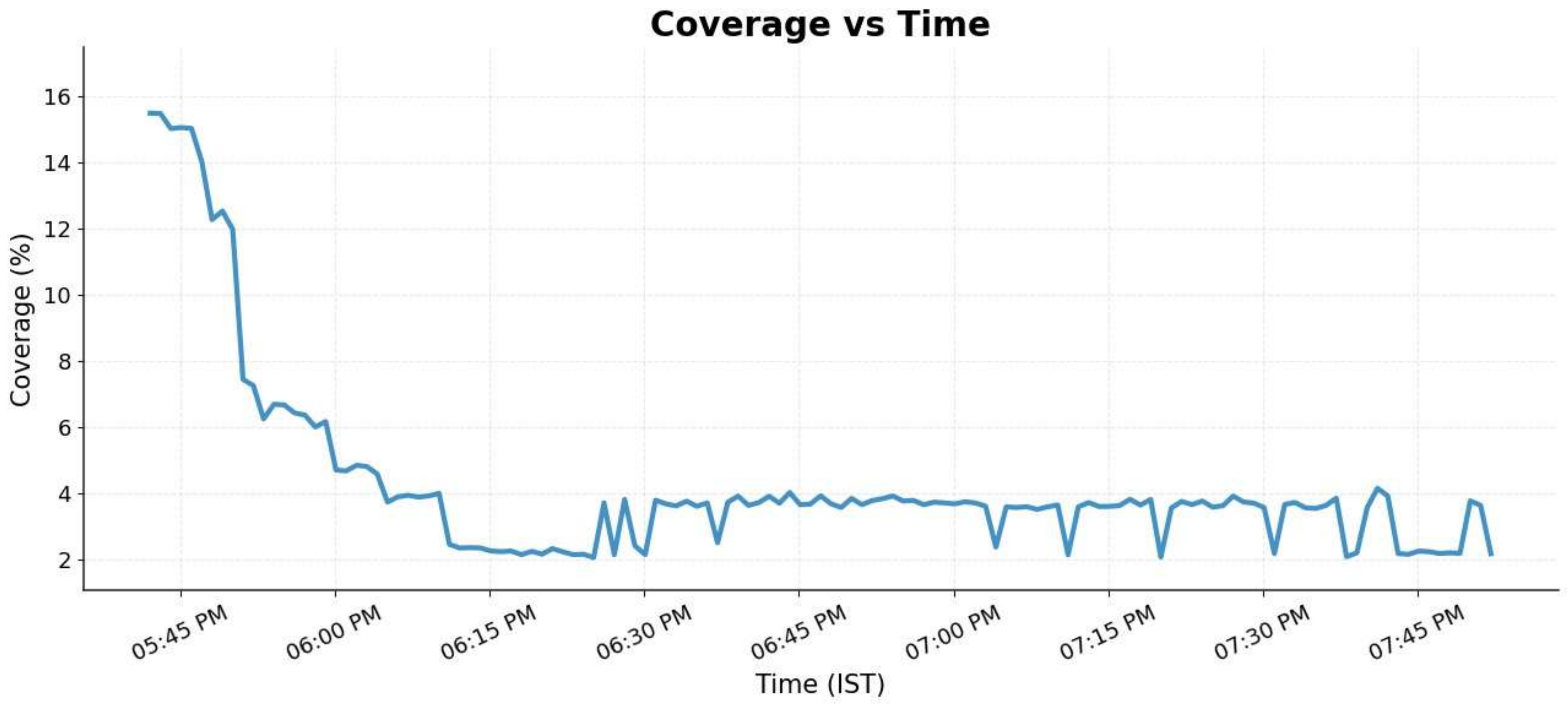
- The Raspberry Pi 4 has the YOLOv11 model uploaded in it
- It has a separate web interface shown using RealVNC
- In the interface, we can run the model on the local host and can see the segmented image along with the per class pixel count.
- It also shows the processing time



Deployment

- We deployed our model inside the campus to test its functioning
- We left it in the field for a several hours and provided a continuous power supply

Deployment Analysis



Potential Future Developments

- Predictive analysis based on human and animal activity
- Enhanced classification and waste categorisation
- Integration with IoT ecosystems
- 5G connectivity integration for real time streaming and cloud analytics

THANK YOU!